

PRÁCTICA FINAL

Autor: José María Gómez Laynez

Asignatura: Computación Ubicua e Inteligencia Ambiental

Fecha: 27/05/2022

Documentación

En mi proyecto para esta asignatura he realizado una aplicación pensada para utilizar en floristerías. En concreto, la aplicación tiene tres funcionalidades: una para capturar audio, otra que analiza marcadores y otra de reconocimiento facial.

El funcionamiento tipo de la aplicación es el siguiente: el cliente entra a la floristería y si no tiene mucha idea de que flor elegir podrá analizar su cara (reconocimiento e identificación de imágenes) y recibir una recomendación de flor según sus parámetros faciales, también podrá ver otras recomendaciones. Si el cliente no sabe el aspecto de esta flor, el florista le dará un marcador con el modelo de la flor para que el cliente pueda verla (realidad aumentada). La navegación por la aplicación podrá ser con comandos de voz (reconocimiento de voz) o simplemente usando el teclado del ordenador.

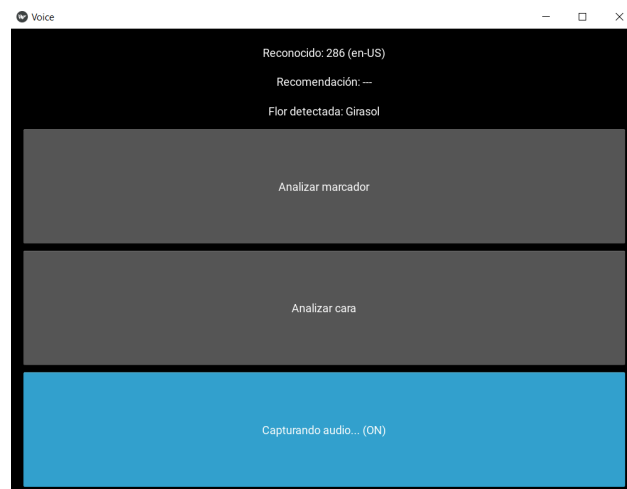
Manual del usuario

Al iniciar la app se muestra una interfaz realizada con Kivy, que dispone de tres botones (uno para cada funcionalidad). Además, en la parte de arriba de la interfaz se encuentran tres etiquetas que indican el estado del micrófono (si está activo o no), la recomendación de flor en base al reconocimiento facial y la flor detectada en el análisis de marcadores.



El botón de capturar voz es un botón de dos posiciones, si se encuentra activo podremos ejecutar todas las funciones de la aplicación con comandos de voz, en

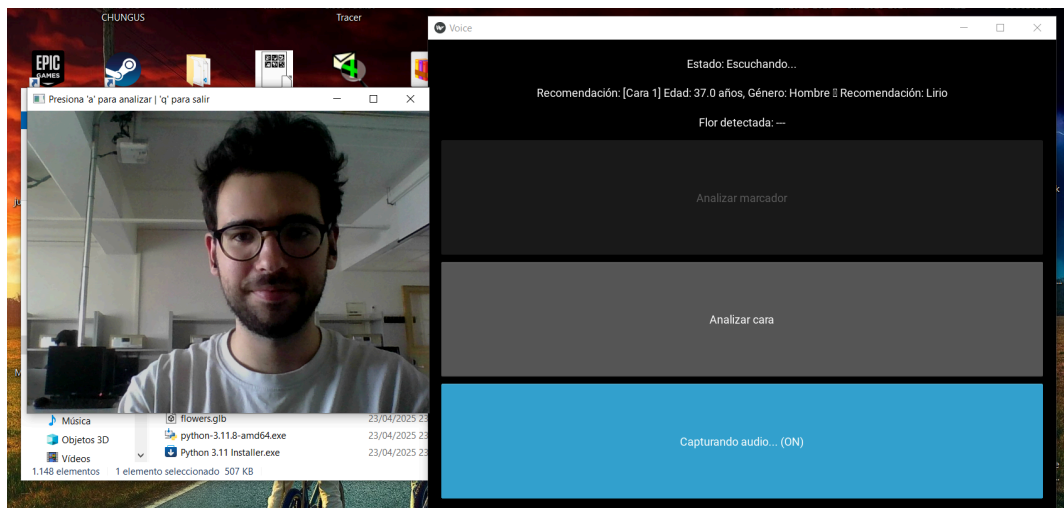
caso contrario se realizará con el teclado. La detección de audio se ha realizado con la herramienta de SpeechRecognition.



Para entrar en la función de analizar marcador, se puede iniciar pinchando en esta o con el audio activo diciendo “Analizar marcador”, “Analyze marker” o “Analyser marqueur”. Una vez que hemos iniciado esta opción se encenderá nuestra cámara y podremos enseñar a ésta los distintos marcadores de los que disponemos (el florista nos dispondrá de 6 marcadores Aruco 5x5). Los marcadores numerados del 0 al 5 corresponden a los siguientes modelos de flores.



Para analizar la cara puede hacerse pulsando el botón de “Analizar cara” o al igual que antes, con el audio activado decir “Analizar cara”, “Analyze face” o “Analyser visage”. Una vez que hemos iniciado el proceso podemos capturar nuestra cara pulsando la tecla “a” o diciendo “capturar”, “capture” o “capturer” y la aplicación nos recomendará una flor en base a los parámetros de edad y género. Para detección de caras se ha utilizado la herramienta de insightface de FaceAnalysis.

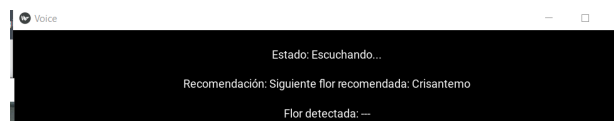


```
def recomendar_flores(edad, genero):
    """
    Devuelve una lista ordenada de flores recomendadas según edad y género.
    """
    recomendaciones = []

    if edad < 13:
        recomendaciones = ["Margarita", "Girasol", "Tulipán"]
    elif edad < 20:
        recomendaciones = ["Girasol", "Margarita", "Rosa"]
    elif edad < 35:
        recomendaciones = ["Rosa", "Tulipán", "Girasol"] if genero == "Mujer" else ["Tulipán", "Girasol", "Rosa"]
    elif edad < 50:
        recomendaciones = ["Tulipán", "Lirio", "Rosa"] if genero == "Mujer" else ["Lirio", "Crisantemo", "Tulipán"]
    elif edad < 70:
        recomendaciones = ["Lirio", "Crisantemo", "Tulipán"] if genero == "Mujer" else ["Crisantemo", "Lirio", "Girasol"]
    else:
        recomendaciones = ["Crisantemo", "Lirio", "Tulipán"]

    return recomendaciones
```

Si pulsamos la tecla “s” nos recomendará otra flor en caso de que no nos guste la elección o también si decimos “next” o “siguiente”. Para salir de esta función habrá que pulsar la tecla “q” o decir “escape”.



Dependencias

Mi proyecto lo he creado en un entorno virtual de Python en la versión Python 3.11.8. Todas las dependencias que he instalado para el funcionamiento correcto de la aplicación han sido estas:

absl-py	2.2.2
albucore	0.0.24
albumentations	2.0.6
annotated-types	0.7.0
anyio	4.9.0
argon2-cffi	23.1.0
argon2-cffi-bindings	21.2.0
arrow	1.3.0
asttokens	3.0.0

astunparse	1.6.3
async-lru	2.0.5
attrs	25.3.0
babel	2.17.0
beautifulsoup4	4.13.4
bleach	6.2.0
blinker	1.9.0
cachetools	5.5.2
certifi	2025.1.31
cffi	1.17.1
charset-normalizer	3.4.1
click	8.1.8
colorama	0.4.6
coloredlogs	15.0.1
comm	0.2.2
contourpy	1.3.2
cycler	0.12.1
Cython	3.1.0
dataclasses-json	0.6.7
debugpy	1.8.14
decorator	5.2.1
defusedxml	0.7.1
docutils	0.21.2
easydict	1.13
executing	2.2.0
fastjsonschema	2.21.1
filelock	3.18.0
filetype	1.2.0
fire	0.7.0
Flask	3.1.0
flask-cors	5.0.1
flatbuffers	25.2.10
fonttools	4.57.0
fqdn	1.5.1
freetype-py	2.5.1
fsspec	2025.3.2
gast	0.4.0
gdown	5.2.0
glfw	1.0.13
google-auth	2.40.0
google-auth-oauthlib	1.0.0
google-pasta	0.2.0
grpcio	1.71.0
gunicorn	23.0.0
h11	0.14.0
h5py	3.13.0
hsluv	5.0.4
httpcore	1.0.8

httpx	0.28.1
huggingface-hub	0.31.2
humanfriendly	10.0
idna	3.10
imageio	2.37.0
insightface	0.7.3
ipykernel	6.29.5
ipython	9.1.0
ipython_pygments_lexers	1.1.1
isoduration	20.11.0
itsdangerous	2.2.0
jedi	0.19.2
Jinja2	3.1.6
joblib	1.5.0
json5	0.12.0
jsonpointer	3.0.0
jsonschema	4.23.0
jsonschema-specifications	2025.4.1
jupyter_client	8.6.3
jupyter_core	5.7.2
jupyter-events	0.12.0
jupyter-lsp	2.2.5
jupyter_server	2.15.0
jupyter_server_terminals	0.5.3
jupyterlab	4.4.1
jupyterlab_pygments	0.3.0
jupyterlab_server	2.27.3
keras	2.13.1
Kivy	2.3.1
kivy-deps.angle	0.4.0
kivy-deps.glew	0.3.1
kivy_deps.gstreamer	0.3.4
kivy_deps.sdl2	0.8.0
Kivy-Garden	0.1.5
kiwisolver	1.4.8
langdetect	1.0.9
lazy_loader	0.4
libclang	18.1.1
lz4	4.4.4
Markdown	3.8
markdown-it-py	3.0.0
MarkupSafe	3.0.2
marshmallow	3.26.1
matplotlib	3.10.1
matplotlib-inline	0.1.7
mdurl	0.1.2
mistune	3.1.3
ml-dtypes	0.4.1

mpmath	1.3.0
mtcnn	1.0.0
mypy_extensions	1.1.0
namex	0.0.9
nbclient	0.10.2
nbconvert	7.16.6
nbformat	5.10.4
nest-asyncio	1.6.0
networkx	3.4.2
notebook_shim	0.2.4
numpy	2.2.5
nvidia-cuda-runtime-cu12	12.9.37
nvidia-pyindex	1.0.9
oauthlib	3.2.2
onnx	1.18.0
onnxruntime	1.22.0
opencv-contrib-python	4.11.0.86
opencv-python	4.11.0.86
opt_einsum	3.4.0
optree	0.15.0
overrides	7.7.0
packaging	25.0
pandas	2.2.3
pandocfilters	1.5.1
parso	0.8.4
pillow	10.4.0
pip	25.0.1
platformdirs	4.3.7
prettytable	3.16.0
prometheus_client	0.21.1
prompt_toolkit	3.0.51
protobuf	4.25.7
psutil	7.0.0
pure_eval	0.2.3
pyasn1	0.6.1
pyasn1_modules	0.4.2
PyAudio	0.2.14
pycparser	2.22
pydantic	2.11.4
pydantic_core	2.33.2
pygfx	0.9.0
Pygments	2.19.1
pylinalg	0.6.7
pyparsing	3.2.3
pypiwin32	223
pyreadline3	3.5.4
PySocks	1.7.1
python-dateutil	2.9.0.post0

python-json-logger	3.3.0
pytz	2025.2
pywin32	310
pywinpty	2.0.15
PyYAML	6.0.2
pyzmq	26.4.0
referencing	0.36.2
regex	2024.11.6
rendercanvas	2.0.4
requests	2.32.3
requests-oauthlib	2.0.0
retina-face	0.0.17
rfc3339-validator	0.1.4
rfc3986-validator	0.1.1
rich	14.0.0
rpds-py	0.24.0
rsa	4.9.1
safetensors	0.5.3
scikit-image	0.25.2
scikit-learn	1.6.1
scipy	1.15.3
Send2Trash	1.8.3
setuptools	65.5.0
simpleaudio	1.0.4
simsimd	6.2.1
six	1.17.0
sniffio	1.3.1
soupsieve	2.7
SpeechRecognition	3.14.2
stack-data	0.6.3
stringzilla	3.12.5
sympy	1.14.0
tensorboard	2.13.0
tensorboard-data-server	0.7.2
tensorflow	2.13.1
tensorflow-estimator	2.13.0
tensorflow-intel	2.13.1
tensorflow-io-gcs-filesystem	0.31.0
tensorrt-cu12	10.10.0.31
tensorrt_cu12_bindings	10.10.0.31
tensorrt_cu12_libs	10.10.0.31
termcolor	3.1.0
terminado	0.18.1
threadpoolctl	3.6.0
tifffile	2025.5.10
tinycss2	1.4.0
tokenizers	0.21.1
tornado	6.4.2

tqdm	4.67.1
traitlets	5.14.3
transformers	4.51.3
types-python-dateutil	2.9.0.20241206
typing_extensions	4.13.2
typing-inspect	0.9.0
typing-inspection	0.4.0
tzdata	2025.2
uharfbuzz	0.49.0
uri-template	1.3.0
urllib3	2.4.0
wcwidth	0.2.13
webcolors	24.11.1
webencodings	0.5.1
websocket-client	1.8.0
Werkzeug	3.1.3
wgpu	0.21.1
wheel	0.45.1
wrapt	1.14.1

Para obtener estos datos he usado el comando de “pip list” para ver todas las dependencias de mi entorno. Las más importantes son: opencv-python para cv2, numpy, insightface para FaceAnalysis, kivy para la interfaz, matplotlib, pygfx para el renderizado gráfico, pylinalg y SpeechRecognition.

Archivos

Los archivos para el funcionamiento de la práctica son:

- Modelos glb:
 - crisantemo.glb
 - girasol.glb
 - lirio_2.glb
 - margarita.glb
 - rosa.glb
 - tulipan.glb
- Código de la asignatura:
 - [cuia.py](#): contiene algunas funciones básicas de opencv, cámara, modelos...
 - [camara.py](#): archivo generado automáticamente al calibrar la cámara
- Código mío:
 - [cara.py](#): para el análisis de caras.
 - [voz.py](#): se encarga de capturar la voz y los comandos
 - [marcador.py](#): contiene el código para cargar los modelos y usarlos en los marcadores
 - [main.py](#): controla la interfaz de la aplicación y el flujo de esta

- `estado_compartido.py`: contiene algunas variables compartidas entre archivos, sobre todo booleanos.